Université de Gafsa
Institut Supérieur des Sciences Appliquées et de Technologie de Gafsa
Département Informatique

Conception et Développement d'un Système Automatisé d'Auto-Défense Réseau (ANSDS)

Entreprise d'accueil
Agence Nationale de la Cybersécurité (ANCS)

Auteur : Yahya Imen

Encadrant professionnel : M. Bengayed Wajdi

Année universitaire 2025–2026

Remerciements

Je tiens à exprimer ma profonde gratitude à l'Agence Nationale de la Cybersécurité (ANCS) pour m'avoir accueilli durant ce stage et m'avoir offert un environnement propice à l'apprentissage et à la réalisation de ce projet.

Mes sincères remerciements s'adressent particulièrement à mon encadrant professionnel, M. Bengayed Wajdi, pour son accompagnement, ses conseils, sa disponibilité et la confiance qu'il m'a accordée tout au long de ce travail.

Je remercie également l'Institut Supérieur des Sciences Appliquées et de Technologie de Gafsa, ainsi que l'ensemble des enseignants du Département Informatique, pour la qualité de la formation reçue.

Enfin, j'adresse mes remerciements à toutes les personnes ayant contribué, de près ou de loin, à la réussite de ce projet de stage.

Résumé

Ce rapport présente la conception et le développement d'un système automatisé d'auto-défense réseau intitulé **Automated Network Self-Defense System (ANSDS)**. Ce projet a été réalisé dans le cadre d'un stage d'été effectué à l'Agence Nationale de la Cybersécurité (ANCS).

L'objectif principal est de concevoir un système capable de détecter automatiquement des attaques SSH, d'envoyer une alerte en temps réel via Telegram, puis de mettre en œuvre une réponse automatique en bloquant l'adresse IP de l'attaquant sur un routeur Cisco à l'aide de la bibliothèque Netmiko.

Une plateforme de test a été développée sous GNS3 en intégrant un routeur Cisco, un commutateur, un serveur Ubuntu, une machine Kali Linux ainsi qu'un accès Internet. Cette infrastructure permet de reproduire des scénarios d'attaque et de valider les différentes phases du système.

Les développements futurs portent sur l'analyse du trafic réseau avec Wireshark, la détection des attaques, l'automatisation de la réponse, puis la validation complète du système.

Mots-clés : Cybersécurité, SSH, GNS3, Cisco IOS, Netmiko, Wireshark, Python, Auto-Défense Réseau.

Abstract

This report presents the design and development of an automated network self-defense system called **Automated Network Self-Defense System (ANSDS)**. The project was carried out during a summer internship at the National Cybersecurity Agency (ANCS).

The main objective is to design a system capable of automatically detecting SSH attacks, sending real-time alerts through Telegram, and automatically blocking the attacker's IP address on a Cisco router using the Netmiko library.

A laboratory environment was developed using GNS3, including a Cisco router, a Cisco switch, an Ubuntu Server, a Kali Linux machine, and Internet connectivity. This platform makes it possible to reproduce attack scenarios and validate each phase of the project.

Future work includes SSH traffic analysis with Wireshark, attack detection, automated response mechanisms, and complete system validation.

Keywords : Cybersecurity, SSH, GNS3, Cisco IOS, Netmiko, Wireshark, Python, Network Self-Defense.

Table des matières

1 Introduction Générale **1**

2 Conception du Laboratoire Réseau **2**
2.1 Introduction . . . 2
2.2 Objectifs du laboratoire . . . 2
2.3 Architecture générale du laboratoire . . . 3
2.4 Équipements du laboratoire . . . 3
2.4.1 Routeur Cisco . . . 4
2.4.2 Commutateur Cisco . . . 5
2.4.3 Serveur Ubuntu . . . 5
2.4.4 Machine Kali Linux . . . 5
2.5 Plan d'adressage IP . . . 5
2.6 Configuration NAT . . . 6
2.7 Validation de l'environnement réseau . . . 7
2.7.1 Version Cisco IOS . . . 7
2.7.2 Validation de la connectivité Internet . . . 7
2.8 Validation des communications réseau . . . 7
2.8.1 Communication entre le serveur Ubuntu et le routeur . . . 7
2.8.2 Communication entre Kali Linux et le routeur . . . 9
2.8.3 Communication entre le serveur Ubuntu et Kali Linux . . . 9
2.8.4 Communication entre Kali Linux et le serveur Ubuntu . . . 9
2.9 Validation du service SSH . . . 10
2.9.1 Installation du serveur OpenSSH . . . 10
2.9.2 Vérification de l'état du service . . . 11
2.9.3 Vérification du port d'écoute . . . 11
2.9.4 Validation d'une connexion SSH . . . 11
2.10 Conclusion . . . 11

3 Développement du moteur de détection SSH avec Scapy **13**
3.1 Introduction . . . 13
3.2 Objectifs de la phase . . . 13
3.3 Étude du protocole SSH . . . 14
3.3.1 Établissement d'une connexion SSH . . . 14
3.3.2 Analyse du Three-Way Handshake TCP . . . 14
3.3.3 Informations observables malgré le chiffrement . . . 16
3.4 Développement progressif du moteur de détection . . . 17
3.4.1 Première capture réseau avec Scapy . . . 17
3.4.2 Filtrage du trafic SSH . . . 18

3.4.3 Détection des paquets TCP SYN 19
3.4.4 Comptage des tentatives de connexion 20
3.4.5 Détection par seuil 20
3.4.6 Fenêtre temporelle glissante 20
3.5 Validation expérimentale 21
3.6 Analyse des résultats 22
3.7 Conclusion 22

Table des figures

2.1 Architecture générale du laboratoire réseau développé sous GNS3 3
2.2 Configuration matérielle du routeur Cisco 4
2.3 Configuration IP des interfaces du routeur Cisco 4
2.4 Configuration réseau du serveur Ubuntu 5
2.5 Configuration réseau de la machine Kali Linux 6
2.6 Configuration NAT du routeur Cisco 6
2.7 Vérification de la version Cisco IOS 8
2.8 Validation de la connectivité Internet 8
2.9 Test de connectivité entre le serveur Ubuntu et le routeur Cisco 9
2.10 Test de connectivité entre Kali Linux et le routeur Cisco 9
2.11 Communication entre le serveur Ubuntu et Kali Linux 10
2.12 Communication entre Kali Linux et le serveur Ubuntu 10
2.13 Installation du serveur OpenSSH 11
2.14 Vérification de l'état du service SSH 11
2.15 Vérification de l'écoute du port TCP 22 11
2.16 Connexion SSH réussie entre Kali Linux et le serveur Ubuntu 12

3.1 Vue générale d'une connexion SSH 15
3.2 Paquet TCP SYN 15
3.3 Paquet TCP SYN/ACK 16
3.4 Paquet TCP ACK 16
3.5 Première version du détecteur développé avec Scapy 18
3.6 Capture des paquets réseau avec Scapy 18
3.7 Filtrage des paquets SSH 19
3.8 Détection des paquets TCP SYN 19
3.9 Comptage des tentatives de connexion 20
3.10 Déclenchement de l'alerte après dépassement du seuil 20
3.11 Détection d'une attaque après application de la fenêtre temporelle 21

Liste des tableaux

2.1 Plan d'adressage IP du laboratoire 6

Chapitre 1

Introduction Générale

Ce chapitre présentera le contexte, la problématique, les objectifs, la méthodologie ainsi que l'organisation générale du rapport.

Chapitre 2

Conception du Laboratoire Réseau

2.1 Introduction

Avant de développer le système **Automated Network Self-Defense System (ANSDS)**, il a été indispensable de concevoir une plateforme de test permettant de reproduire un environnement réseau réaliste et entièrement maîtrisé. Ce laboratoire constitue la base expérimentale du projet et permet de développer, de tester et de valider chaque composant du système dans des conditions proches d'une infrastructure d'entreprise.

L'environnement a été réalisé à l'aide de GNS3 en intégrant un routeur Cisco IOS, un commutateur Ethernet, un serveur Ubuntu, une machine Kali Linux ainsi qu'un accès Internet via un mécanisme de traduction d'adresses réseau (Network Address Translation – NAT). Cette architecture offre un environnement adapté à l'étude du protocole SSH, à la capture des paquets réseau ainsi qu'au développement des mécanismes de détection et de réponse automatique.

Avant de commencer les phases de détection d'attaques, l'ensemble des équipements a été configuré puis validé afin de garantir le bon fonctionnement des communications réseau.

2.2 Objectifs du laboratoire

La mise en place du laboratoire poursuit plusieurs objectifs techniques indispensables au développement du projet ANSDS :

- Concevoir une infrastructure réseau représentative d'un environnement réel.
- Déployer un serveur Ubuntu jouant le rôle de machine victime.
- Déployer une machine Kali Linux représentant l'attaquant.
- Configurer le routage et l'accès Internet à l'aide d'un routeur Cisco.
- Mettre en œuvre la traduction d'adresses (NAT).
- Valider l'ensemble des communications réseau avant le développement des mécanismes de sécurité.
- Préparer un environnement permettant l'analyse du trafic SSH avec Wireshark.
- Fournir une plateforme stable pour les futures phases de détection et de réponse automatique.

2.3 Architecture générale du laboratoire

L'architecture du laboratoire a été développée sous GNS3 afin de reproduire une infrastructure réseau simple, stable et facilement reproductible.

Elle est constituée des éléments suivants :

— un routeur Cisco IOS assurant le routage des paquets et l'accès Internet ;
— un commutateur Cisco reliant les différentes machines du réseau local ;
— un serveur Ubuntu représentant la machine victime ;
— une machine Kali Linux représentant l'attaquant ;
— un nœud NAT permettant l'accès aux ressources Internet.

La Figure 2.1 présente l'architecture générale du laboratoire utilisée durant la totalité de la première phase du projet.

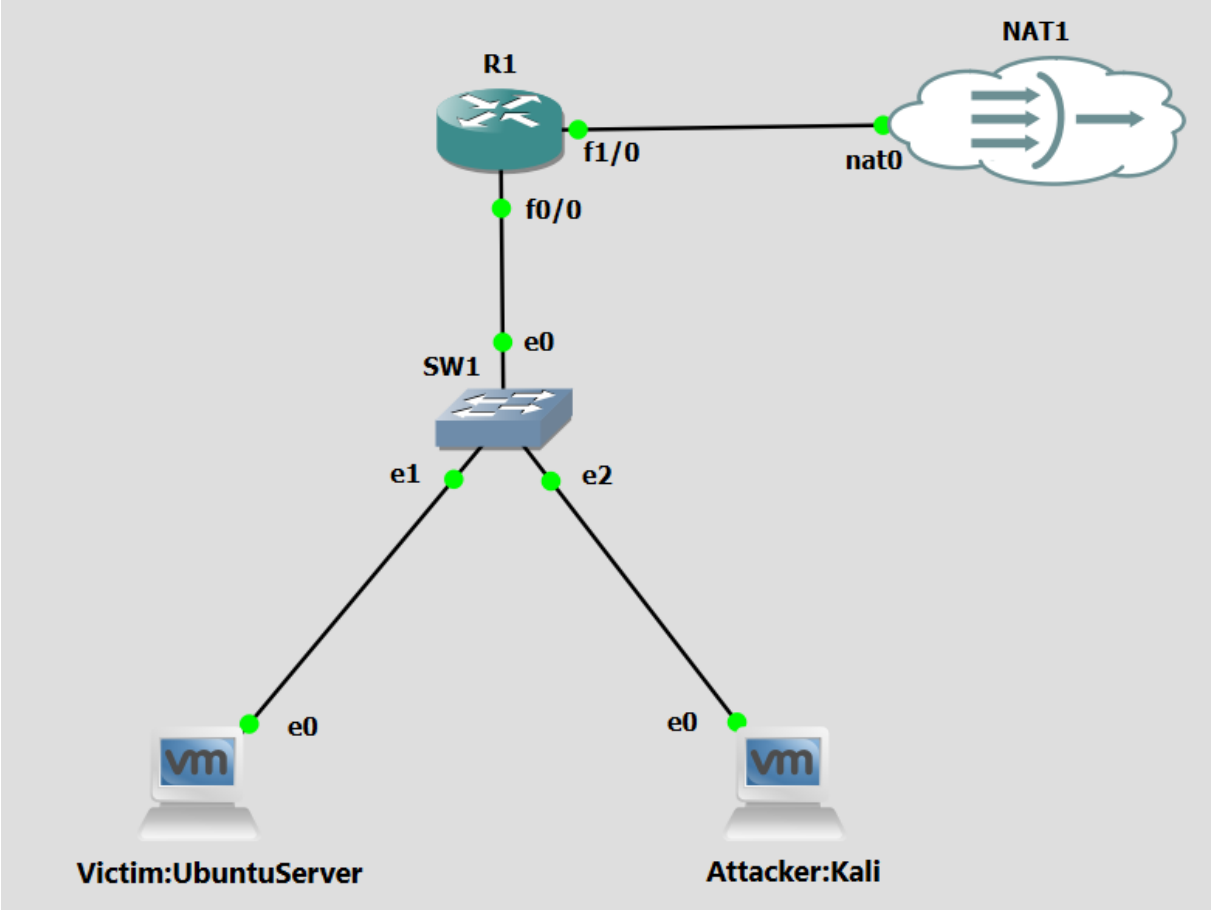


FIGURE 2.1 – Architecture générale du laboratoire réseau développé sous GNS3

Cette architecture permet d'isoler totalement l'environnement de test tout en offrant les fonctionnalités nécessaires au développement du système ANSDS. Elle constitue la base sur laquelle seront réalisées les captures réseau, les attaques SSH simulées ainsi que les mécanismes de détection et de défense présentés dans les chapitres suivants.

2.4 Équipements du laboratoire

L'infrastructure du laboratoire repose sur plusieurs équipements réseau ayant chacun une fonction bien définie. Ensemble, ils permettent de reproduire un environnement repré-

sentatif d'un réseau d'entreprise tout en offrant une plateforme adaptée au développement et à la validation du système ANSDS.

Les sous-sections suivantes présentent le rôle de chaque équipement ainsi que les principales configurations réalisées.

2.4.1 Routeur Cisco

Le routeur Cisco constitue l'élément central de l'architecture réseau. Il assure le routage des paquets entre le réseau local et Internet, ainsi que la mise en œuvre du mécanisme de traduction d'adresses (NAT). Il jouera également un rôle essentiel dans les phases suivantes du projet, puisqu'il sera chargé d'appliquer automatiquement des listes de contrôle d'accès (ACL) afin de bloquer les adresses IP identifiées comme malveillantes.

Le routeur utilisé est basé sur la plateforme Cisco 7200 et dispose de plusieurs interfaces FastEthernet permettant d'interconnecter le réseau local au nœud NAT de GNS3.

La Figure 2.2 présente la configuration matérielle retenue pour le routeur.

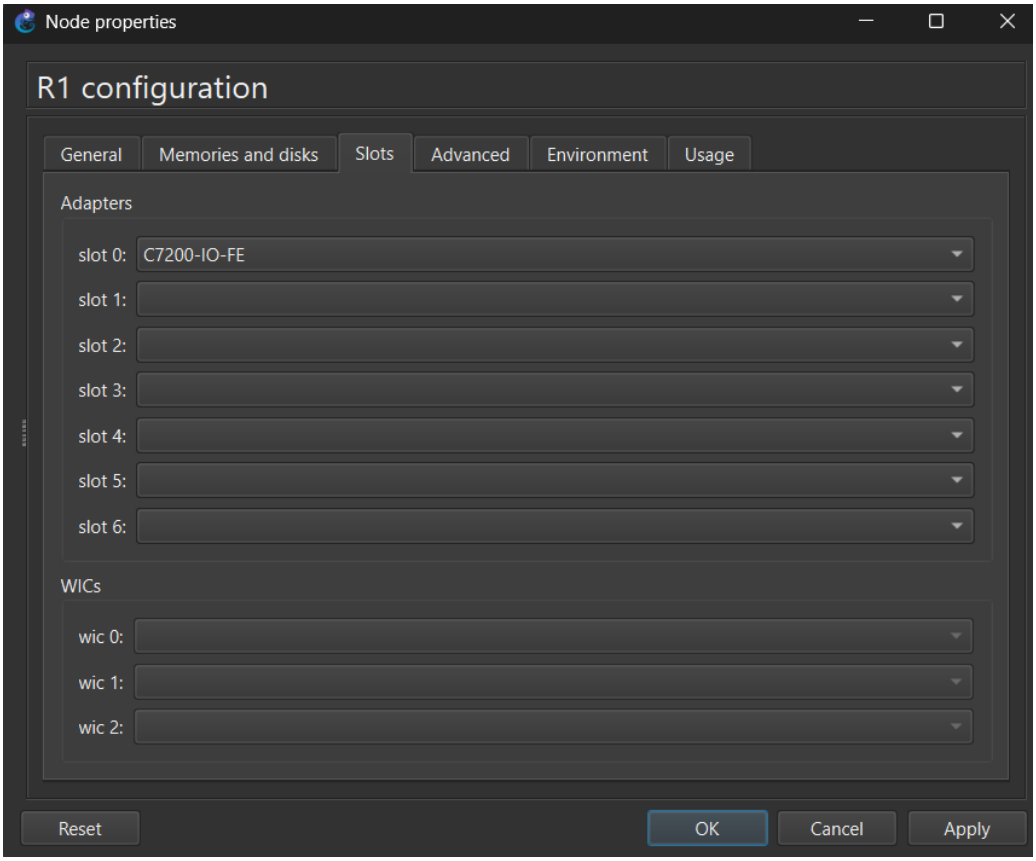


FIGURE 2.2 – Configuration matérielle du routeur Cisco

La configuration IP des interfaces du routeur est présentée à la Figure 2.3. L'interface FastEthernet0/0 est utilisée comme passerelle du réseau local tandis que l'interface FastEthernet1/0 assure la connexion vers Internet.

```
R1#show ip int br
Interface                  IP-Address      OK? Method Status                Protocol
FastEthernet0/0            192.168.10.1    YES manual up                    up
```

FIGURE 2.3 – Configuration IP des interfaces du routeur Cisco

2.4.2 Commutateur Cisco

Le commutateur Cisco assure l'interconnexion des équipements du réseau local. Il relie le serveur Ubuntu, la machine Kali Linux et le routeur Cisco au sein du même segment Ethernet.

Aucune configuration spécifique n'a été nécessaire sur ce commutateur. Son rôle est exclusivement de fournir la connectivité de niveau 2 entre les différents équipements du laboratoire. Comme présenté à la Figure 2.1, il constitue le point central des communications locales.

2.4.3 Serveur Ubuntu

Le serveur Ubuntu représente la machine victime du projet. Il héberge le service OpenSSH qui sera utilisé lors des différentes expérimentations de sécurité. Les futures attaques par force brute ainsi que les mécanismes de détection développés dans ANSDS cibleront cette machine.

Une configuration IP statique a été appliquée afin de garantir la stabilité de l'environnement de test. Le serveur utilise l'adresse IP 192.168.10.10/24 et la passerelle par défaut 192.168.10.1.

La configuration réseau du serveur est illustrée à la Figure 2.4.

```
imen@ubuntuserver:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:09:36:59 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.10.10/24 brd 192.168.10.255 scope global ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe09:3659/64 scope link
       valid_lft forever preferred_lft forever
imen@ubuntuserver:~$ ip route
default via 192.168.10.1 dev ens33 proto static
192.168.10.0/24 dev ens33 proto kernel scope link src 192.168.10.10
```

FIGURE 2.4 – Configuration réseau du serveur Ubuntu

2.4.4 Machine Kali Linux

La machine Kali Linux représente l'attaquant dans le laboratoire. Elle est utilisée pour établir les connexions SSH, générer les attaques simulées et valider le fonctionnement des mécanismes de détection et de réponse automatique.

Comme le serveur Ubuntu, cette machine possède une configuration IP statique afin d'assurer la reproductibilité des expérimentations. Son adresse IP est 192.168.10.20/24 et sa passerelle par défaut est 192.168.10.1.

La Figure 2.5 présente la configuration réseau de la machine Kali Linux.

2.5 Plan d'adressage IP

Afin d'assurer une communication stable entre les différents équipements du laboratoire, un plan d'adressage IP statique a été défini. Chaque équipement possède une adresse IP fixe facilitant les phases de développement, de test et de validation.

Le Tableau 2.1 présente le plan d'adressage adopté.

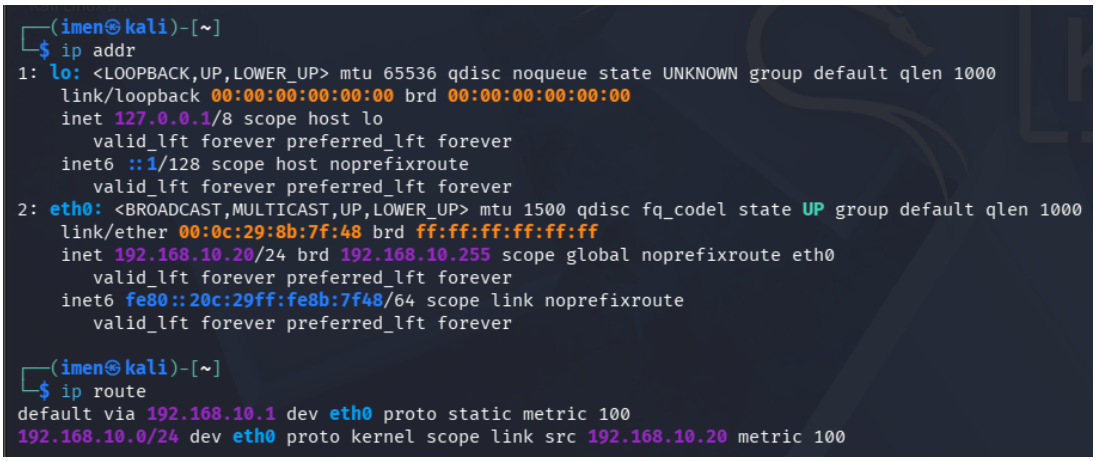


FIGURE 2.5 – Configuration réseau de la machine Kali Linux

TABLE 2.1 – Plan d'adressage IP du laboratoire

Équipement	Interface	Adresse IP	Passerelle
Routeur Cisco	FastEthernet0/0	192.168.10.1/24	—
Serveur Ubuntu	ens33	192.168.10.10/24	192.168.10.1
Kali Linux	eth0	192.168.10.20/24	192.168.10.1

L'ensemble des machines du réseau local appartient au sous-réseau `192.168.10.0/24`. Le routeur Cisco joue le rôle de passerelle par défaut et assure l'acheminement des paquets vers le réseau externe via le mécanisme de traduction d'adresses (NAT).

2.6 Configuration NAT

Afin de permettre aux machines virtuelles du laboratoire d'accéder au réseau Internet, une traduction d'adresses réseau (*Network Address Translation* – NAT) a été configurée sur le routeur Cisco.

Le mécanisme retenu est le *Port Address Translation* (PAT), également appelé surcharge NAT (*NAT Overload*). Cette technique permet à plusieurs machines du réseau privé de partager une seule adresse IP publique correspondant à l'interface connectée au réseau externe.

Dans cette configuration, l'interface FastEthernet0/0 est définie comme interface interne (*inside*), tandis que l'interface FastEthernet1/0 est configurée comme interface externe (*outside*).

La Figure 2.6 présente la configuration NAT appliquée sur le routeur Cisco.

```
R1#show run | section ip nat
 ip nat inside
 ip nat outside
ip nat inside source list 1 interface FastEthernet1/0 overload
R1#
```

FIGURE 2.6 – Configuration NAT du routeur Cisco

L'instruction `ip nat inside source list 1 interface FastEthernet1/0 overload` active le mécanisme PAT. Toutes les machines appartenant à la liste d'accès numéro 1 peuvent ainsi accéder au réseau externe en utilisant l'adresse IP de l'interface FastEthernet1/0.

Cette configuration constitue une étape essentielle du laboratoire, puisqu'elle garantit la connectivité Internet nécessaire à l'installation des paquets logiciels, aux mises à jour du système ainsi qu'aux différentes expérimentations réalisées durant le projet.

2.7 Validation de l'environnement réseau

Une fois la configuration des différents équipements terminée, plusieurs vérifications ont été réalisées afin de s'assurer du bon fonctionnement de l'environnement avant de débuter les expérimentations liées au système ANSDS.

Ces vérifications concernent notamment la version du système Cisco IOS ainsi que la connectivité Internet du laboratoire.

2.7.1 Version Cisco IOS

Le routeur utilisé dans le laboratoire exécute le système d'exploitation Cisco IOS. La vérification de la version installée permet de confirmer la compatibilité des fonctionnalités utilisées dans le cadre du projet, notamment le routage IP, la traduction d'adresses (NAT) et les listes de contrôle d'accès (ACL).

La Figure 2.7 présente les informations relatives à la version du système Cisco IOS.

2.7.2 Validation de la connectivité Internet

Après la configuration du mécanisme NAT, un test de connectivité a été réalisé afin de vérifier que le routeur était capable d'accéder au réseau Internet.

La réussite de ce test confirme le bon fonctionnement du routage ainsi que de la traduction d'adresses configurée précédemment.

La Figure 2.8 présente le résultat de ce test.

La validation de ces deux éléments confirme que l'infrastructure réseau est correctement configurée et prête pour les phases de test et de développement du système ANSDS.

2.8 Validation des communications réseau

Une fois l'ensemble des équipements configurés, plusieurs tests de connectivité ont été réalisés afin de vérifier le bon fonctionnement de l'infrastructure réseau. Ces tests permettent de s'assurer que chaque machine peut communiquer correctement avec les autres équipements du laboratoire avant le déploiement des mécanismes de sécurité.

Les essais ont été réalisés à l'aide de la commande `ping`, qui permet de vérifier la disponibilité d'un équipement ainsi que la bonne configuration du routage.

2.8.1 Communication entre le serveur Ubuntu et le routeur

Le premier test consiste à vérifier que le serveur Ubuntu peut joindre la passerelle du réseau local.

```
R1#show version
Cisco IOS Software, 7200 Software (C7200-ADVIPSERVICESK9-M), Version 15.0(1)M, RELEASE SOFTWARE (fc2)
Technical Support: http://www.cisco.com/techsupport
Copyright (c) 1986-2009 by Cisco Systems, Inc.
Compiled Wed 30-Sep-09 07:48 by prod_rel_team

ROM: ROMMON Emulation Microcode
BOOTLDR: 7200 Software (C7200-ADVIPSERVICESK9-M), Version 15.0(1)M, RELEASE SOFTWARE (fc2)

R1 uptime is 2 minutes
System returned to ROM by unknown reload cause - suspect boot_data[BOOT_COUNT] 0x0, BOOT_COUNT 0, BOOTDATA 19
System image file is "tftp://255.255.255.255/unknown"


This product contains cryptographic features and is subject to United
States and local country laws governing import, export, transfer and
use. Delivery of Cisco cryptographic products does not imply
third-party authority to import, export, distribute or use encryption.
Importers, exporters, distributors and users are responsible for
compliance with U.S. and local country laws. By using this product you
agree to comply with applicable laws and regulations. If you are unable
to comply with U.S. and local laws, return this product immediately.

A summary of U.S. laws governing Cisco cryptographic products may be found at:
http://www.cisco.com/wwl/export/crypto/tool/stqrg.html

If you require further assistance please contact us by sending email to
export@cisco.com.

Cisco 7206VXR (NPE400) processor (revision A) with 491520K/32768K bytes of memory.
Processor board ID 4279256517
R7000 CPU at 150MHz, Implementation 39, Rev 2.1, 256KB L2 Cache
6 slot VXR midplane, Version 2.1

Last reset from power-on

PCI bus mb0_mb1 (Slots 0, 1, 3 and 5) has a capacity of 600 bandwidth points.
Current configuration on bus mb0_mb1 has a total of 600 bandwidth points.
This configuration is within the PCI bus capacity and is supported.

PCI bus mb2 (Slots 2, 4, 6) has a capacity of 600 bandwidth points.
Current configuration on bus mb2 has a total of 0 bandwidth points
This configuration is within the PCI bus capacity and is supported.

Please refer to the following document "Cisco 7200 Series Port Adaptor
Hardware Configuration Guidelines" on Cisco.com <http://www.cisco.com>
```

FIGURE 2.7 – Vérification de la version Cisco IOS

```
R1#ping 8.8.8.8

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 8.8.8.8, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 76/84/108 ms
R1#
```

FIGURE 2.8 – Validation de la connectivité Internet

La Figure 2.9 montre que le routeur répond correctement aux requêtes ICMP envoyées par le serveur Ubuntu.

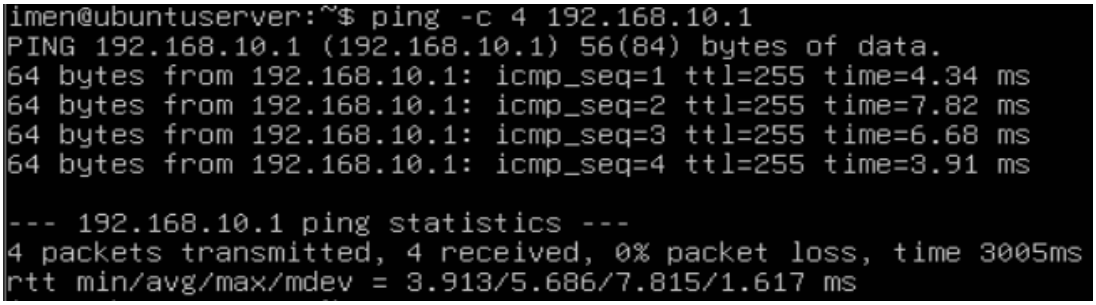


FIGURE 2.9 – Test de connectivité entre le serveur Ubuntu et le routeur Cisco

2.8.2 Communication entre Kali Linux et le routeur

Le même test a été réalisé depuis la machine Kali Linux afin de vérifier sa connectivité avec la passerelle du réseau.

Comme illustré à la Figure 2.10, les réponses ICMP confirment le bon fonctionnement de cette liaison.

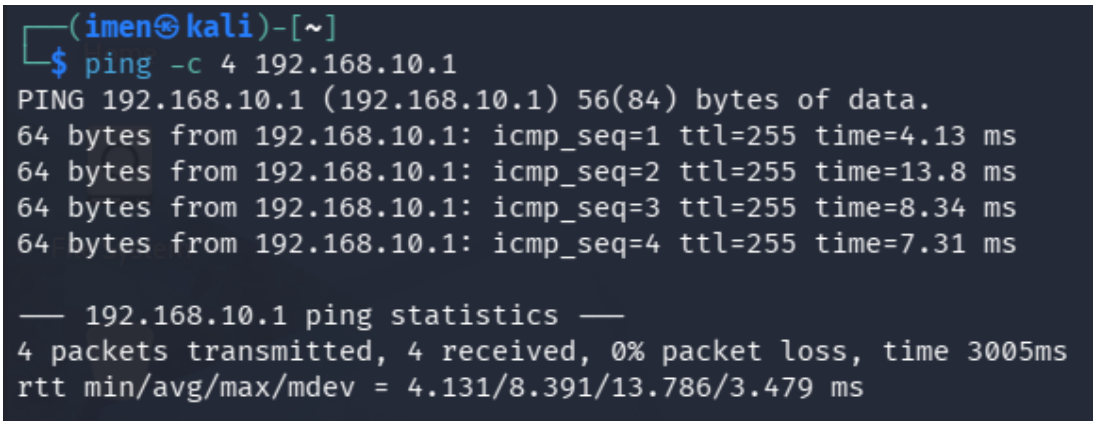


FIGURE 2.10 – Test de connectivité entre Kali Linux et le routeur Cisco

2.8.3 Communication entre le serveur Ubuntu et Kali Linux

Une communication directe entre les deux machines virtuelles a ensuite été vérifiée.

La Figure 2.11 montre que le serveur Ubuntu peut communiquer correctement avec la machine Kali Linux.

2.8.4 Communication entre Kali Linux et le serveur Ubuntu

Enfin, le test inverse a été réalisé afin de confirmer la connectivité bidirectionnelle entre les deux machines.

Les résultats présentés à la Figure 2.12 confirment que les échanges réseau sont fonctionnels dans les deux sens.

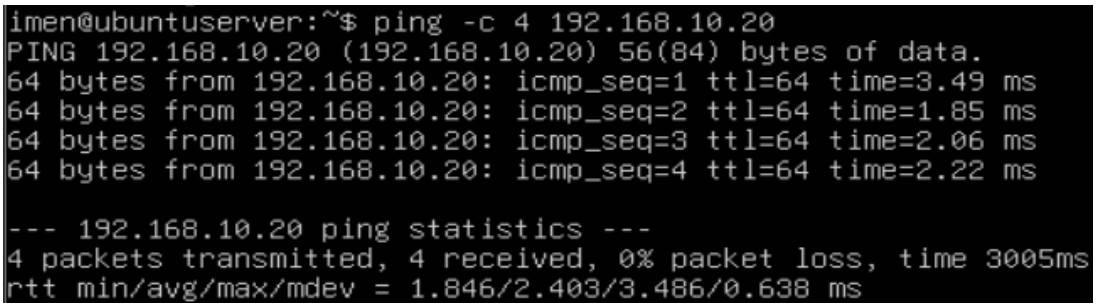


FIGURE 2.11 – Communication entre le serveur Ubuntu et Kali Linux

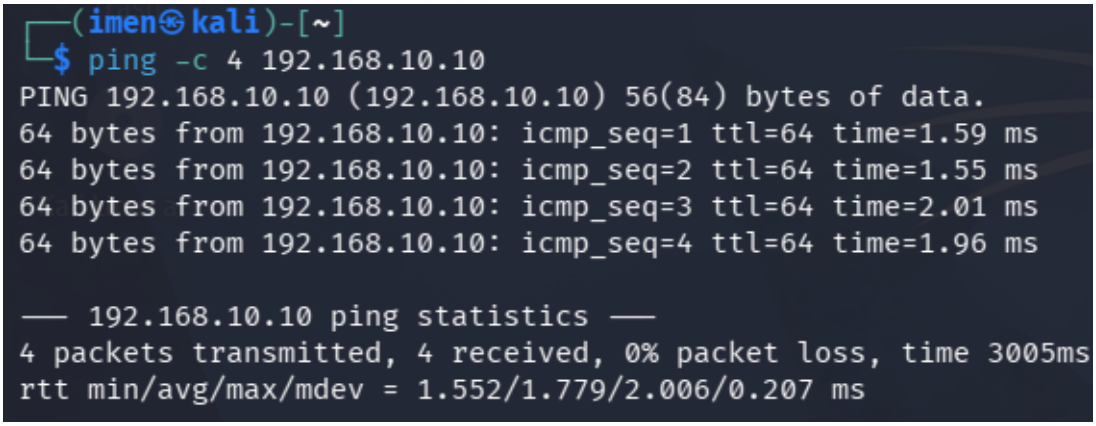


FIGURE 2.12 – Communication entre Kali Linux et le serveur Ubuntu

Les différents tests de connectivité réalisés confirment le bon fonctionnement de l'infrastructure réseau. Le routage IP, les configurations des interfaces ainsi que les communications entre les différents équipements ont été validés avec succès. Cette étape garantit que le laboratoire est prêt pour l'installation et les tests du service SSH, qui constituent la base des expérimentations présentées dans la suite du projet.

2.9 Validation du service SSH

Le protocole Secure Shell (SSH) constitue l'élément central des expérimentations réalisées dans le cadre du projet ANSDS. Avant de développer les mécanismes de détection et de réponse automatique, il est indispensable de vérifier que le service SSH est correctement installé, actif et accessible depuis les autres équipements du laboratoire.

Plusieurs vérifications successives ont été réalisées afin de valider le bon fonctionnement du service.

2.9.1 Installation du serveur OpenSSH

Le serveur Ubuntu a été configuré afin d'héberger le service OpenSSH. Cette étape permet d'accepter les connexions SSH provenant de la machine Kali Linux.

La Figure 2.13 présente l'installation du serveur OpenSSH.



FIGURE 2.13 – Installation du serveur OpenSSH

2.9.2 Vérification de l'état du service

Une fois l'installation terminée, le statut du service SSH a été vérifié afin de confirmer son bon fonctionnement.

Comme l'illustre la Figure 2.14, le service est correctement démarré et prêt à accepter les connexions entrantes.

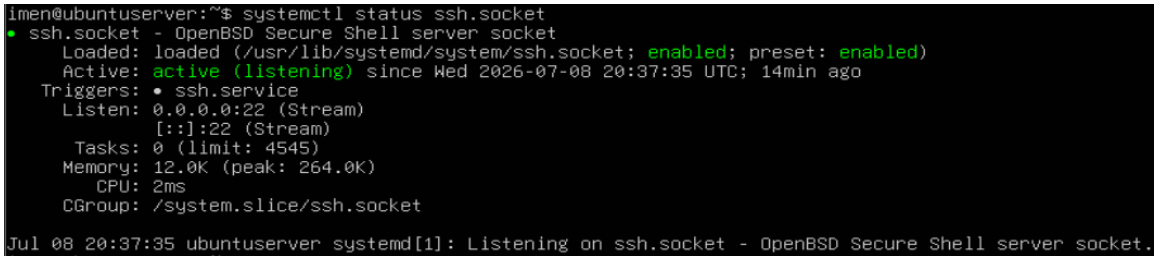


FIGURE 2.14 – Vérification de l'état du service SSH

2.9.3 Vérification du port d'écoute

Le fonctionnement du serveur SSH a également été validé en vérifiant que le port TCP 22 est en état d'écoute.

La Figure 2.15 confirme que le service écoute correctement sur le port standard utilisé par le protocole SSH.

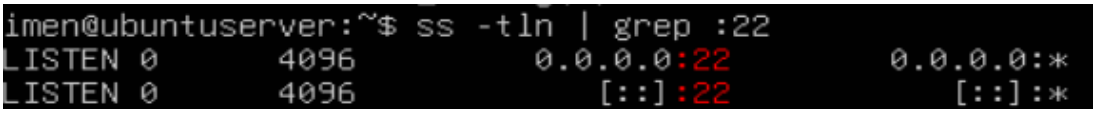


FIGURE 2.15 – Vérification de l'écoute du port TCP 22

2.9.4 Validation d'une connexion SSH

La dernière étape consiste à établir une connexion SSH entre la machine Kali Linux et le serveur Ubuntu.

La réussite de cette connexion confirme que l'ensemble des configurations réseau et du service SSH sont opérationnelles. Cette validation est essentielle, car toutes les expérimentations des chapitres suivants reposeront sur cette communication.

La Figure 2.16 présente une connexion SSH réussie entre les deux machines.

2.10 Conclusion

Au cours de ce chapitre, un laboratoire réseau complet a été conçu et configuré afin de servir de plateforme d'expérimentation pour le projet ANSDS. L'architecture mise en



FIGURE 2.16 – Connexion SSH réussie entre Kali Linux et le serveur Ubuntu

place comprend un routeur Cisco IOS, un commutateur Ethernet, un serveur Ubuntu, une machine Kali Linux ainsi qu'un accès Internet via un mécanisme NAT.

Les différentes étapes de configuration ont été validées par des tests de connectivité réseau et par la vérification du fonctionnement du service SSH. Ces résultats confirment que l'environnement expérimental est pleinement opérationnel et constitue une base fiable pour le développement des mécanismes de détection des attaques et de réponse automatique.

Le chapitre suivant sera consacré à l'analyse du trafic SSH à l'aide de Wireshark. Cette étude permettra de comprendre le déroulement d'une connexion SSH, d'identifier les paquets échangés et de préparer la mise en œuvre des mécanismes de détection utilisés dans le système ANSDS.

Chapitre 3

Développement du moteur de détection SSH avec Scapy

3.1 Introduction

Après la mise en place de l'infrastructure réseau présentée dans le chapitre précédent ainsi que l'étude détaillée du protocole SSH réalisée à l'aide de Wireshark, la phase suivante du projet consiste à développer le premier composant actif du système **Automated Network Self-Defense System (ANSDS)**.

L'objectif de cette phase est de concevoir un moteur capable d'observer le trafic réseau en temps réel afin d'identifier automatiquement des comportements pouvant correspondre au début d'une attaque SSH. Contrairement aux approches reposant sur l'analyse des journaux d'authentification (*logs*), le détecteur développé dans ce projet s'appuie directement sur l'observation des paquets réseau. Cette approche permet de détecter les tentatives de connexion dès leur établissement, avant même que le serveur SSH ne traite les demandes d'authentification.

Pour réaliser cette détection, la bibliothèque Python **Scapy** a été retenue. Cet outil offre des fonctionnalités avancées de capture, d'analyse et de manipulation de paquets réseau tout en permettant le développement rapide d'algorithmes de détection personnalisés. Son intégration en Python facilite également l'évolution progressive du système vers des mécanismes plus avancés tels que les notifications automatiques ou le blocage dynamique des adresses IP malveillantes.

Le développement du détecteur a été réalisé de manière incrémentale. Chaque fonctionnalité a été implémentée, testée puis validée avant de passer à l'étape suivante. Cette méthode permet de limiter les erreurs, de faciliter les phases de débogage et de garantir la fiabilité du composant développé.

Dans ce chapitre, nous présentons successivement les objectifs poursuivis, les principes de fonctionnement du détecteur, les différentes étapes de son développement, les validations expérimentales réalisées ainsi que les résultats obtenus.

3.2 Objectifs de la phase

Cette phase constitue la première étape du développement du moteur de détection d'intrusion du projet ANSDS.

Les objectifs principaux sont les suivants :

— mettre en place un environnement Python dédié au développement du détecteur ;
— installer et configurer la bibliothèque Scapy afin de permettre la capture des paquets réseau ;
— observer le trafic réseau circulant sur l'interface du serveur Ubuntu ;
— filtrer uniquement le trafic correspondant au protocole SSH (port TCP 22) ;
— identifier les paquets TCP SYN représentant les nouvelles tentatives de connexion ;
— associer chaque tentative à son adresse IP source ;
— compter le nombre de tentatives réalisées par chaque adresse IP ;
— développer un mécanisme de détection basé sur un seuil configurable ;
— améliorer la précision de la détection en utilisant une fenêtre temporelle glissante ;
— valider expérimentalement le fonctionnement de chaque fonctionnalité développée.

À l'issue de cette phase, le système est capable d'identifier automatiquement des séries de tentatives de connexion SSH susceptibles de correspondre au début d'une attaque par force brute. Ce moteur constitue la base fonctionnelle sur laquelle seront ajoutés, dans les phases suivantes, les mécanismes de notification et de réponse automatique.

3.3 Étude du protocole SSH

Avant de développer le moteur de détection, il est indispensable de comprendre précisément le fonctionnement d'une connexion SSH ainsi que les informations qui restent observables malgré le chiffrement des échanges.

Cette étude préalable a été réalisée à l'aide de l'analyseur de protocoles Wireshark. L'objectif n'était pas uniquement d'observer les paquets échangés, mais surtout d'identifier les informations réellement exploitables par le futur moteur de détection développé avec Scapy.

L'analyse a permis de distinguer les différentes étapes de l'établissement d'une connexion SSH, d'identifier les champs réseau accessibles sans déchiffrement et de déterminer les éléments les plus pertinents pour détecter automatiquement des comportements suspects.

3.3.1 Établissement d'une connexion SSH

Une connexion SSH repose sur le protocole TCP. Avant tout échange de données, une connexion fiable doit être établie entre le client et le serveur à l'aide du mécanisme appelé *Three-Way Handshake*.

Une fois cette connexion établie, le client et le serveur échangent leurs informations de version, négocient les algorithmes cryptographiques à utiliser, réalisent l'échange des clés puis activent le chiffrement de la communication.

À partir de cette étape, le contenu des messages devient totalement chiffré. En revanche, plusieurs informations réseau restent visibles, notamment les adresses IP, les ports TCP, les drapeaux TCP, les numéros de séquence ainsi que la fréquence des paquets échangés.

Ces informations constituent la principale source de données exploitée par le moteur de détection développé dans ce projet.

3.3.2 Analyse du Three-Way Handshake TCP

Le Three-Way Handshake représente la première étape de toute connexion SSH.

Cette procédure se déroule selon trois échanges successifs :

1. le client envoie un paquet TCP SYN afin de demander l'ouverture d'une nouvelle connexion ;
2. le serveur répond par un paquet TCP SYN/ACK indiquant qu'il accepte la connexion ;
3. le client termine l'établissement de la connexion en envoyant un paquet TCP ACK.

La Figure 3.1 présente une vue générale d'une connexion SSH observée sous Wireshark.

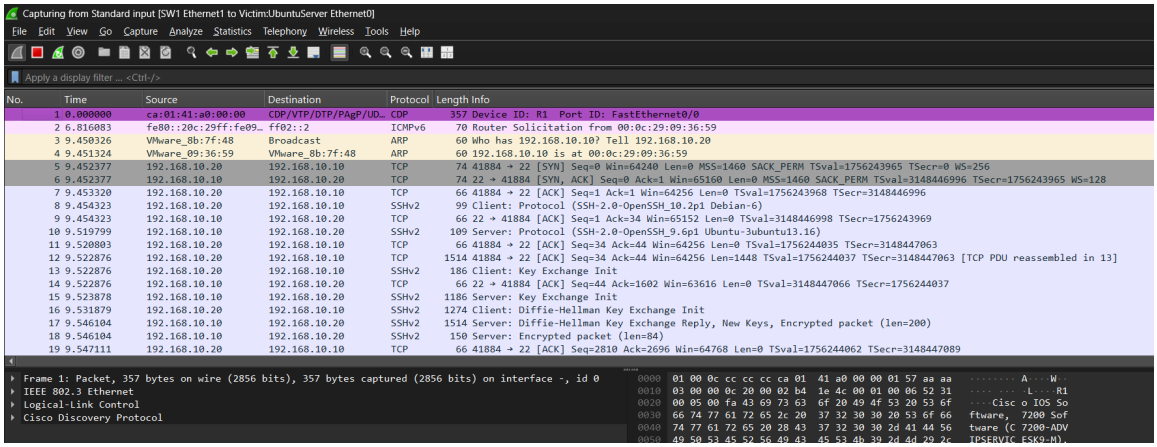


FIGURE 3.1 – Vue générale d'une connexion SSH

Les Figures 3.2, 3.3 et 3.4 présentent respectivement les trois paquets composant cette phase d'établissement.

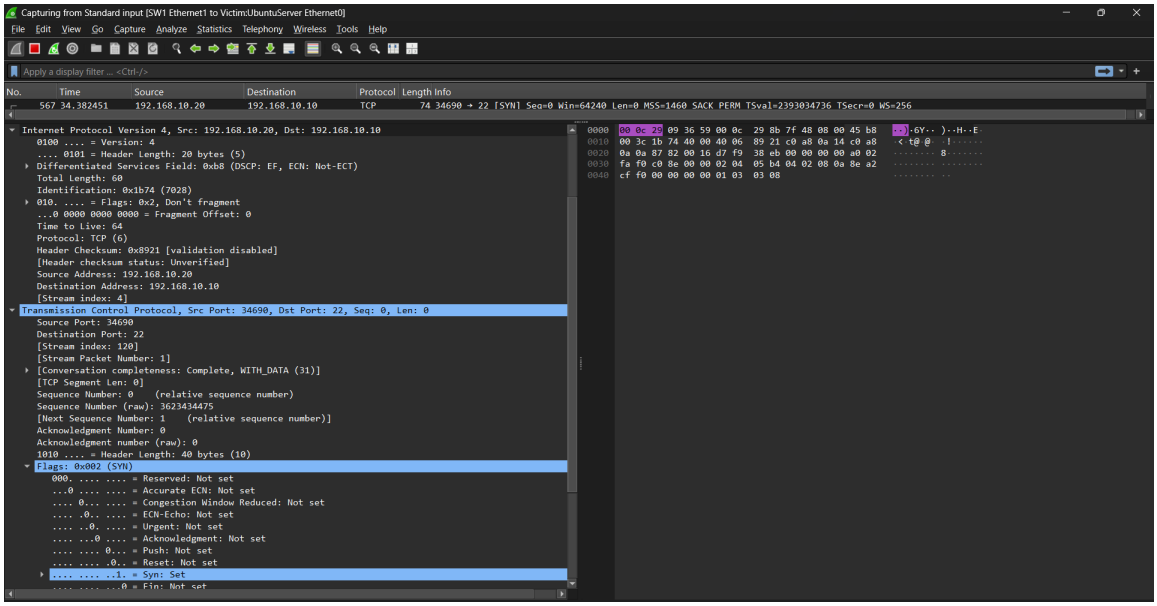


FIGURE 3.2 – Paquet TCP SYN

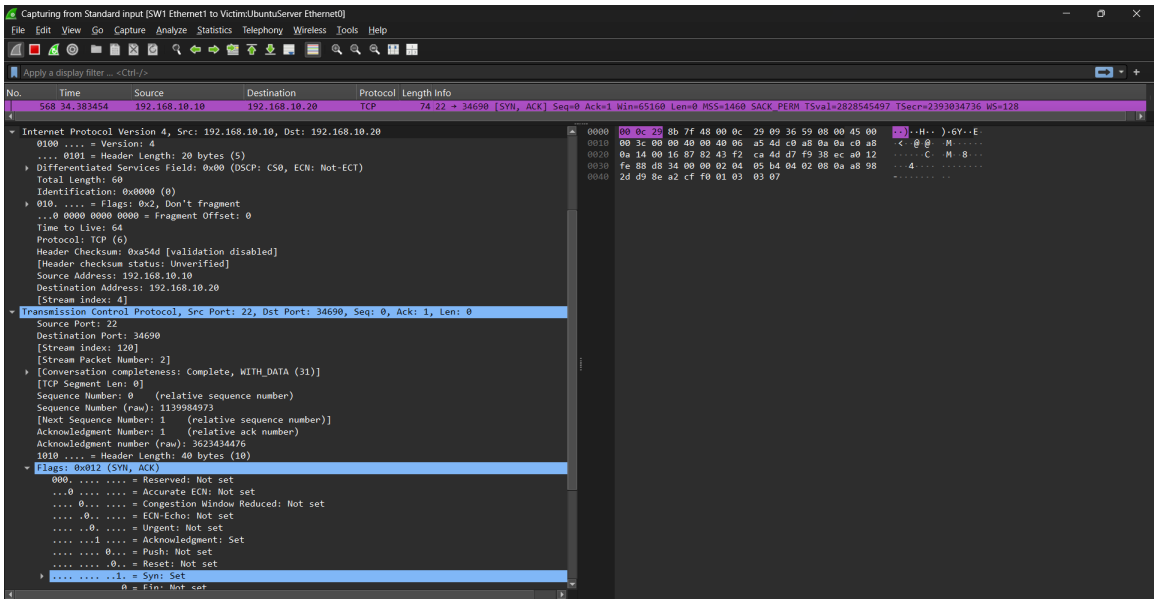


FIGURE 3.3 – Paquet TCP SYN/ACK

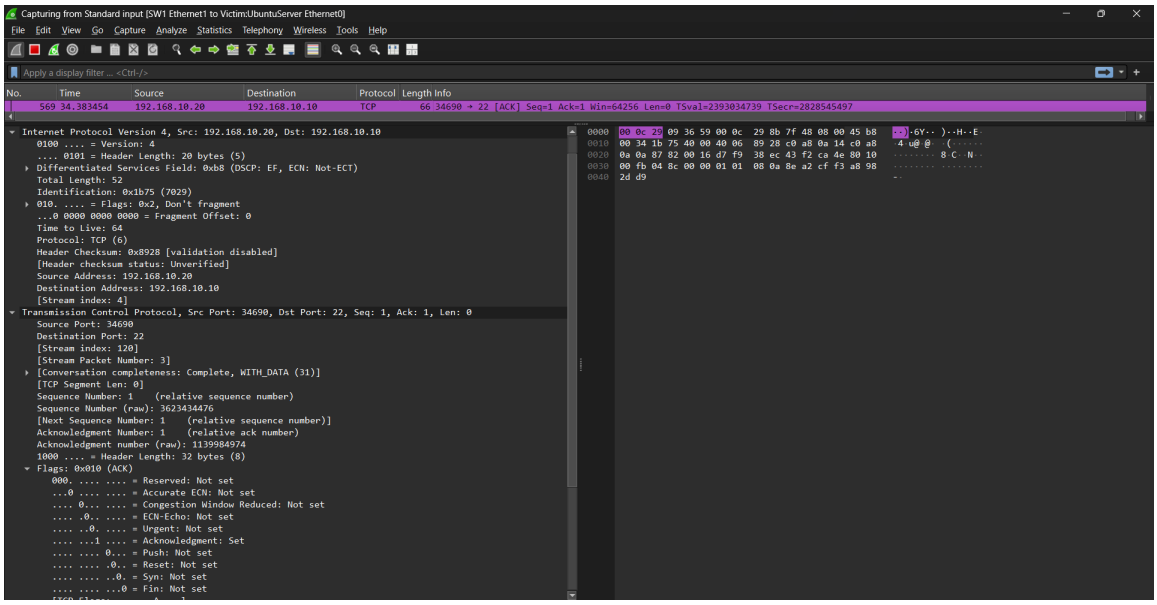


FIGURE 3.4 – Paquet TCP ACK

Parmi ces trois paquets, seul le paquet TCP SYN est utilisé par le moteur de détection. En effet, chaque tentative de connexion SSH débute obligatoirement par l'émission d'un paquet SYN vers le port TCP 22. L'observation de ces paquets permet donc d'identifier rapidement les nouvelles tentatives de connexion initiées par un client.

3.3.3 Informations observables malgré le chiffrement

Après l'établissement de la connexion TCP, le protocole SSH procède à l'échange des informations de version, à la négociation des algorithmes cryptographiques puis à l'échange des clés de chiffrement.

Une fois cette phase terminée, les données applicatives deviennent totalement chiffrées et leur contenu n'est plus directement exploitable.

Cependant, certaines informations restent toujours visibles au niveau des couches réseau et transport :

— les adresses IP source et destination ;
— les ports TCP utilisés ;
— les drapeaux TCP (SYN, ACK, FIN, RST, etc.) ;
— les numéros de séquence TCP ;
— la taille des paquets ;
— la fréquence des connexions ;
— les instants d'émission des paquets.

Ces informations sont largement suffisantes pour construire un moteur de détection basé sur l'analyse comportementale du trafic réseau.

Dans ce projet, la stratégie retenue consiste à surveiller les paquets TCP SYN destinés au port SSH, à associer chaque tentative à son adresse IP source puis à mesurer le nombre de connexions initiées pendant une fenêtre temporelle de dix secondes. Lorsque le nombre de tentatives dépasse le seuil défini, le système considère qu'un comportement anormal est détecté et génère une alerte.

3.4 Développement progressif du moteur de détection

Le développement du moteur de détection a été réalisé de manière incrémentale. Chaque nouvelle fonctionnalité a été implémentée, testée puis validée avant l'ajout de la suivante.

Cette approche présente plusieurs avantages. Elle facilite le débogage, permet de vérifier individuellement chaque composant du détecteur et garantit que les différentes fonctionnalités reposent sur une base technique stable avant leur intégration.

Le développement s'est déroulé en plusieurs étapes successives, allant de la simple capture de paquets jusqu'à la détection d'un comportement caractéristique d'une attaque par force brute sur le service SSH.

3.4.1 Première capture réseau avec Scapy

La première étape du développement consiste à vérifier le bon fonctionnement de la bibliothèque Scapy sur le serveur Ubuntu.

Un premier script Python est développé afin de capturer l'ensemble des paquets circulant sur l'interface réseau `ens33`. À ce stade, aucun filtrage n'est appliqué. L'objectif est uniquement de valider la capacité du programme à intercepter les paquets en temps réel.

La Figure 3.5 présente la première version du script de capture.

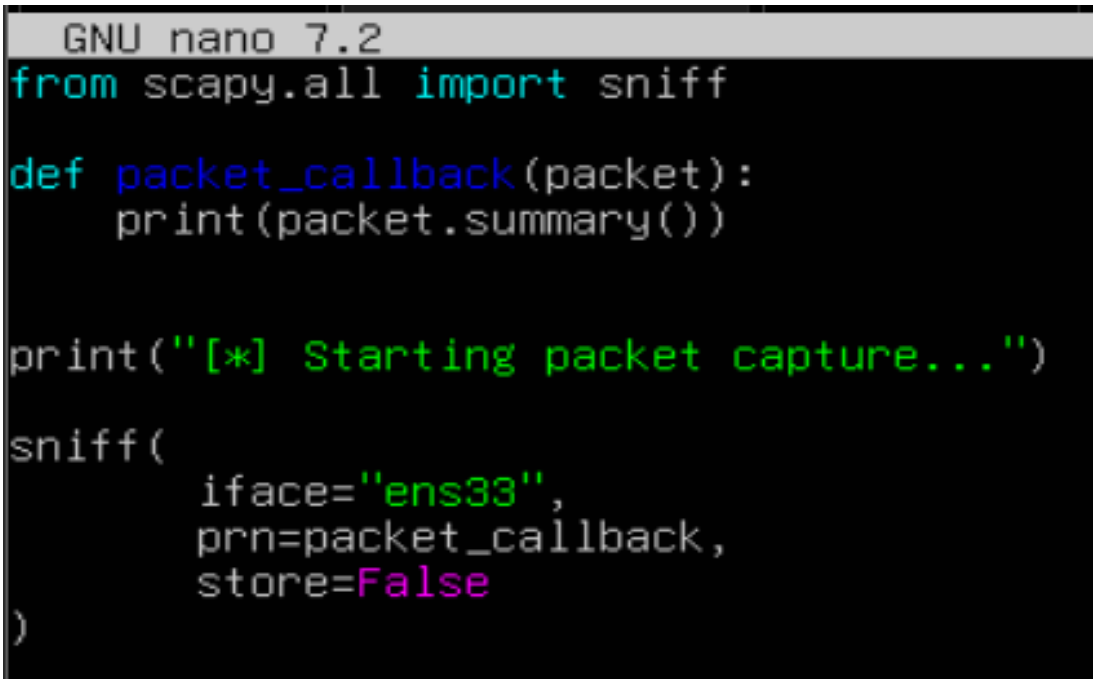


FIGURE 3.5 – Première version du détecteur développé avec Scapy

L'exécution du script confirme que Scapy capture correctement les paquets présents sur le réseau local.

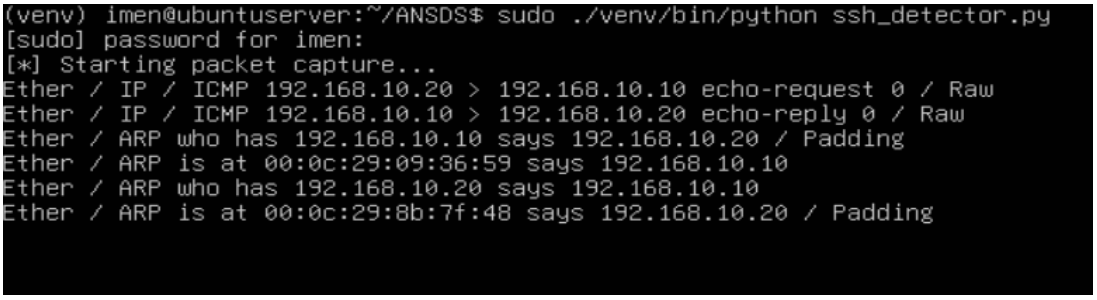


FIGURE 3.6 – Capture des paquets réseau avec Scapy

3.4.2 Filtrage du trafic SSH

La capture de l'ensemble du trafic génère un volume important d'informations ne présentant aucun intérêt pour la détection des attaques SSH.

Afin de réduire le nombre de paquets analysés, un filtre Berkeley Packet Filter (BPF) est ajouté au moteur de capture. Ce filtre limite les paquets capturés au trafic utilisant le port TCP 22.

Cette optimisation réduit considérablement la charge de traitement et concentre l'analyse uniquement sur les connexions SSH.

La Figure 3.7 illustre le résultat obtenu après l'application du filtre.

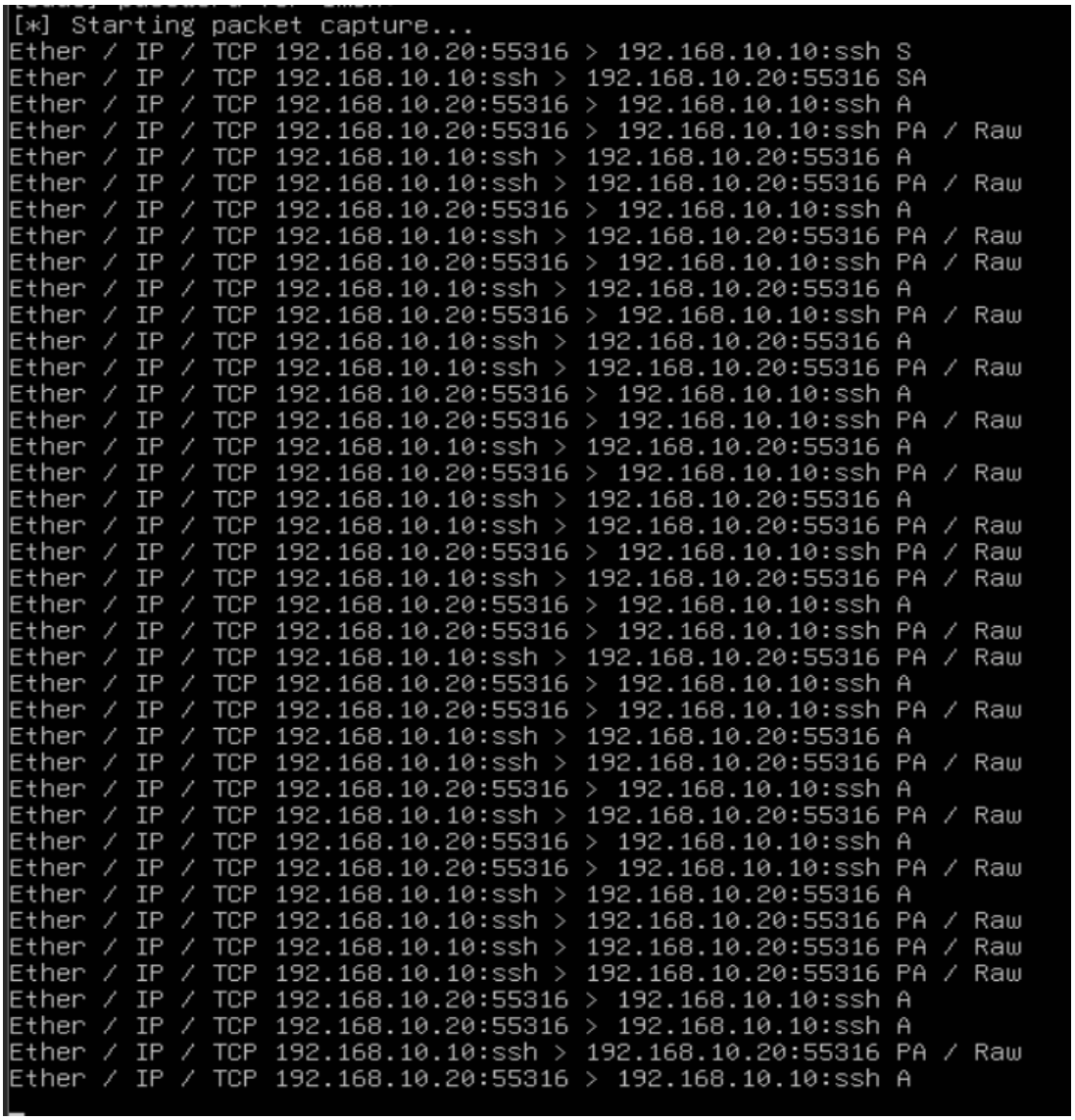


FIGURE 3.7 – Filtrage des paquets SSH

3.4.3 Détection des paquets TCP SYN

Une connexion SSH débute systématiquement par l'émission d'un paquet TCP SYN.

Le moteur de détection est ensuite modifié afin d'identifier uniquement ces paquets. Les drapeaux TCP sont analysés afin de distinguer les nouvelles demandes de connexion des autres paquets composant la communication.

Chaque paquet SYN détecté entraîne l'affichage de l'adresse IP source et de l'adresse IP destination.

Cette étape constitue la première véritable fonctionnalité de détection développée dans le projet.

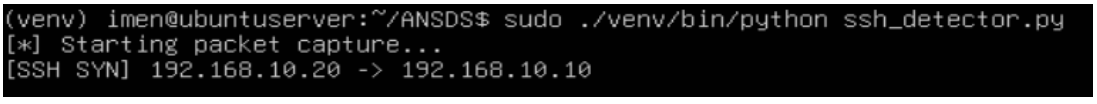


FIGURE 3.8 – Détection des paquets TCP SYN

3.4.4 Comptage des tentatives de connexion

La détection d'un unique paquet SYN ne permet pas de distinguer une connexion normale d'un comportement malveillant.

Le détecteur est donc enrichi par un mécanisme de comptage des tentatives de connexion pour chaque adresse IP source.

À chaque réception d'un paquet TCP SYN, le compteur associé à l'adresse IP émettrice est incrémenté.

Cette approche permet de suivre l'évolution du nombre de connexions initiées par chaque hôte observé sur le réseau.

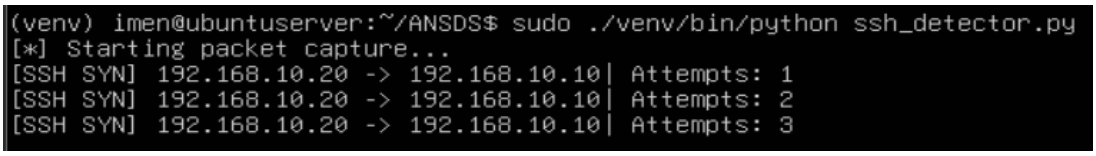


FIGURE 3.9 – Comptage des tentatives de connexion

3.4.5 Détection par seuil

Afin d'identifier automatiquement un comportement anormal, un seuil de détection est introduit.

Dans cette première version du détecteur, une alerte est générée lorsqu'une même adresse IP réalise cinq tentatives de connexion SSH.

Ce mécanisme permet de différencier une connexion isolée d'une succession rapide de tentatives pouvant correspondre au début d'une attaque par force brute.

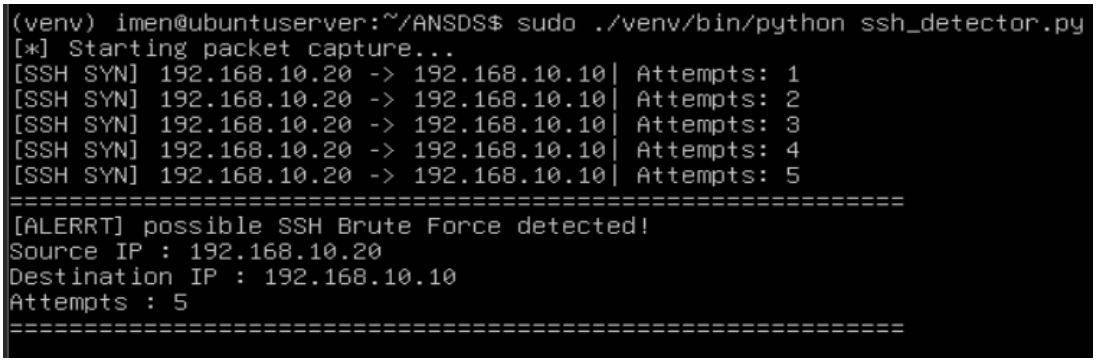


FIGURE 3.10 – Déclenchement de l'alerte après dépassement du seuil

3.4.6 Fenêtre temporelle glissante

L'utilisation d'un compteur cumulatif présente une limite importante : des tentatives espacées de plusieurs minutes peuvent conduire au déclenchement d'une alerte alors qu'elles ne correspondent pas à une attaque.

Afin d'améliorer la précision du détecteur, une fenêtre temporelle glissante de dix secondes est mise en œuvre.

Pour chaque adresse IP, le moteur mémorise l'instant de réception des paquets TCP SYN. Les événements dont l'ancienneté dépasse dix secondes sont automatiquement supprimés avant le calcul du nombre de tentatives.

Ainsi, seules les connexions récentes sont prises en compte pour la prise de décision.

Une alerte est générée uniquement lorsque cinq paquets TCP SYN ou plus sont observés dans cette fenêtre temporelle.

Cette approche réduit significativement les faux positifs et permet une détection plus représentative d'une véritable attaque par force brute.

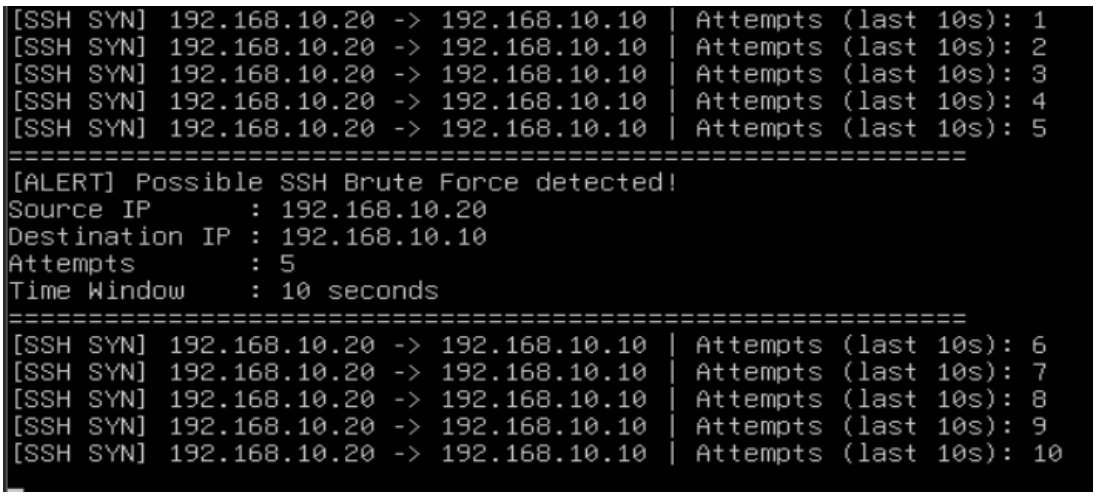


FIGURE 3.11 – Détection d'une attaque après application de la fenêtre temporelle

3.5 Validation expérimentale

Une série de tests a été réalisée afin de vérifier le bon fonctionnement du moteur de détection développé avec Scapy. Chaque fonctionnalité a été validée indépendamment avant son intégration dans le détecteur final.

La première validation a consisté à vérifier la capacité de Scapy à capturer les paquets transitant sur l'interface réseau `ens33`. Après cette étape, un filtre Berkeley Packet Filter (BPF) a été appliqué afin de limiter l'observation aux seuls paquets SSH utilisant le port TCP 22.

Le détecteur a ensuite été configuré pour identifier exclusivement les paquets TCP SYN, correspondant aux nouvelles demandes d'établissement de connexion. Cette fonctionnalité a permis de confirmer que chaque tentative SSH pouvait être détectée dès le début de la communication.

Une phase de validation supplémentaire a consisté à vérifier le mécanisme de comptage des tentatives de connexion par adresse IP. Plusieurs connexions successives ont été initiées depuis la machine Kali Linux afin de contrôler l'incrémentation correcte du compteur.

Enfin, des scénarios simulant une attaque par force brute ont été exécutés. Plusieurs connexions SSH ont été générées dans un intervalle inférieur à dix secondes afin de vérifier le fonctionnement du seuil de détection ainsi que de la fenêtre temporelle glissante.

L'ensemble des essais a confirmé le comportement attendu du détecteur. Les alertes sont uniquement générées lorsque le nombre de tentatives observées dépasse le seuil configuré à l'intérieur de la fenêtre temporelle définie.

3.6 Analyse des résultats

Les expérimentations réalisées montrent que l'utilisation de Scapy constitue une solution adaptée au développement d'un moteur de détection réseau léger et facilement extensible.

L'observation directe des paquets TCP permet de détecter les tentatives de connexion avant même que le serveur SSH ne traite les demandes d'authentification. Cette approche réduit le temps de réaction du système et facilite l'intégration de mécanismes de réponse automatique dans les phases ultérieures du projet.

L'introduction d'un filtrage BPF améliore les performances en limitant l'analyse au trafic réellement pertinent. De plus, l'utilisation d'une fenêtre temporelle glissante améliore significativement la qualité de la détection en évitant que des connexions espacées dans le temps soient interprétées comme une attaque.

Le moteur développé reste volontairement simple afin de privilégier la compréhension des mécanismes fondamentaux de détection. Cette architecture modulaire facilitera l'ajout de nouvelles fonctionnalités telles que la notification Telegram, la journalisation des événements ou encore le blocage automatique des adresses IP via un routeur Cisco.

3.7 Conclusion

Cette phase a permis de développer le premier composant opérationnel du système Automated Network Self-Defense System.

À partir d'une simple capture de paquets réseau, le détecteur a progressivement évolué vers un véritable moteur de détection capable d'identifier des comportements compatibles avec une attaque par force brute sur le service SSH.

Les différentes fonctionnalités développées comprennent la capture des paquets réseau, le filtrage du trafic SSH, la détection des paquets TCP SYN, le comptage des tentatives de connexion, la mise en œuvre d'un seuil de détection ainsi que l'utilisation d'une fenêtre temporelle glissante permettant d'améliorer la précision de l'analyse.

Les résultats obtenus valident les choix techniques retenus et démontrent la faisabilité de la solution proposée.

Le détecteur développé constitue désormais une base solide pour la suite du projet. La phase suivante sera consacrée à l'ajout des mécanismes de réponse automatique, notamment l'envoi d'alertes via Telegram et le blocage dynamique des adresses IP malveillantes sur un routeur Cisco à l'aide de la bibliothèque Netmiko.

Bibliographie